IA Aplicada com LLMs

Aula 15 — O plano de observabilidade

Roteiro

1. **O plano antes da instrumentação** — a lista de métricas como saída
2. **As perguntas** — quem as faz e que decisão informam
3. **O sistema como fluxo de trabalho** — unidade de trabalho e fronteiras
4. **Modos de falha** — e por que os de um agente são silenciosos
5. **Objetivos, sinais e cortes** — SLOs e a matriz de rastreabilidade
6. **Amostragem, qualidade, ação e o que fica fora**
7. **O plano do auditor, e outra arquitetura**

Objetivos

Ao final deste deck, espera-se que o aluno seja capaz de:

- **definir** plano de observabilidade e explicar por que ele precede a instrumentação;
- **derivar** perguntas operacionais dos interessados e descartar as que não informam decisão;
- **modelar** o sistema pela unidade de trabalho, fronteiras de processo e dependências externas;
- **analisar** modos de falha e classificá-los — ruidosos ou silenciosos, abruptos ou de deriva, locais ou sistêmicos;
- **formular** SLOs operacionais e de qualidade e **construir** a matriz de rastreabilidade.

Parte 1

O plano antes da instrumentação

Plano de observabilidade

Definição. Documento que declara, **antes da instrumentação**, que perguntas o sistema responde enquanto opera, que falhas torna visíveis e que objetivos cumpre — e, como consequência, que sinais emite, por quanto tempo os guarda e quem age sobre eles.

Decisão	Artefato
perguntas	lista com interessado e decisão
fluxo de trabalho	diagrama com unidade de trabalho e fronteiras
modos de falha	tabela com classe, sintoma, sinal e ponto de emissão
objetivos	SLIs e SLOs
sinais e cortes	**matriz de rastreabilidade**
amostragem e retenção · qualidade · ação · o que fica fora	políticas declaradas

A lista de métricas é saída do plano, não entrada

- Cada sinal custa emissão, armazenamento e, o mais escasso, **atenção** de quem lê o painel.
- O sinal decisivo num agente raramente vem pronto da plataforma: o **motivo de término** só existe porque foi **projetado** como dado em arquitetura de agentes. Nenhuma instrumentação automática o produziria.
- As quatro métricas da nota sobre observabilidade são o **resultado** do método aplicado ao auditor de despesas.
- Outra arquitetura, com outros modos de falha, chega a **outra lista**.

Parte 2

As perguntas

Pergunta operacional: interessado e decisão

Respondível por consulta à telemetria, e com uma **decisão** que muda conforme a resposta.

Interessado	Natureza	No auditor de despesas
o negócio	entrega o ganho prometido?	a conferência ficou mais rápida na proporção prometida?
a operação	funciona agora? onde quebrou?	quantas análises saíram sem veredito na última hora?
quem paga	quanto custa, e por quê?	qual o custo por despesa, e que componente o domina?
quem responde pelo sistema	agiu dentro do delegado?	alguma análise consultou dado fora da despesa pedida?

Critério de corte. Pergunta sem decisão sai do plano: *quantas vezes por dia* `consultar_politica` *é chamada?* sai; *qual a taxa de erro de* `buscar_regulamento`*?* fica, porque decide a correção da integração.

Parte 3

O sistema como fluxo de trabalho

A unidade de trabalho define o *trace* e o denominador

Unidade de trabalho: a porção do processamento tratada como um todo, do pedido ao resultado.

Arranjo	Unidade	*Trace*
pedido único (o auditor)	a execução	um por execução, `invoke_agent` na raiz
conversa de vários turnos	o turno, agrupado pela conversa	um por turno, com `gen_ai.conversation.id`
confirmação humana	o caso, que pode durar dias	um até a pausa e outro na retomada, ligados por *span link*

Com a pausa dentro do *trace*, o **p95 mede o aprovador**, e não o agente. A espera humana é outra métrica.

Fronteiras, componentes e dependências

- **Fronteiras de processo** — servidor MCP, serviço, outro agente: em cada uma, o contexto do *trace* é **propagado**, ou a árvore se parte em *traces* desconexos.
- **Componentes não determinísticos** — modelo, juiz: falham por **qualidade**.
- **Componentes determinísticos** — ferramenta, banco, índice: falham por **exceção**.
- **Dependências externas** — provedor, APIs de terceiros: falham fora do controle da aplicação, e **mudam sem aviso**.

No auditor: `chat` depende de provedor externo e não determinístico; `execute_tool` e a recuperação do regulamento são determinísticos; término e veredito são **dado da aplicação**.

Parte 4

Modos de falha

FMEA, adaptada à observabilidade

Análise de modos e efeitos de falha (IEC, 2018): para cada componente, as formas de falhar, o efeito e o meio de detecção.

$$(c, m) \mapsto (\text{sintoma},\ \text{sinal},\ \text{ponto de emissão},\ \text{classe})$$

Eixo	Polos	Detecção
manifestação	ruidosa · **silenciosa** (`200` com resultado pior)	a silenciosa não aparece na taxa de erro
evolução	abrupta · **deriva**	a deriva só aparece comparando janelas ou versões
alcance	local · sistêmica	a local exige corte por componente

Linha com **sinal vazio** não é erro da análise: é um achado, e vira requisito de projeto.

Os modos típicos de um agente

Componente	Modo de falha	Classe	Sintoma
laço	esgota o orçamento de passos	silenciosa, deriva	resposta pior, devolvida com sucesso
laço	repete a mesma chamada	silenciosa, abrupta	custo e latência altos
modelo	ferramenta errada, argumento inválido	silenciosa, local	erro de domínio devolvido como observação
modelo	resposta fora do formato	silenciosa, abrupta	resposta sem veredito
modelo	conclui errado	silenciosa, deriva	**nenhum sintoma operacional**
recuperação	trechos vazios ou irrelevantes	silenciosa, deriva	resposta genérica ou recusa
provedor	indisponibilidade ou cota	ruidosa, abrupta	erro de API
provedor	troca o modelo sob o mesmo nome	silenciosa, deriva	acerto muda sem mudar versão
entrada	injeção de prompt	silenciosa, local	ferramenta chamada fora do padrão
histórico	cresce sem compressão	silenciosa, deriva	custo por execução em alta

A tese: as falhas de um agente são silenciosas

- Na tabela, **nove de dez** modos são silenciosos: taxa de erro e latência não os capturam.
- Só **sinais semânticos produzidos pela aplicação** os tornam visíveis:
  - o **motivo de término**;
  - o **veredito** extraído da resposta;
  - o **acerto amostrado**.
- Nenhuma convenção ou plataforma os fornece, porque dependem do que *funcionar* significa no domínio.
- Projetá-los é tarefa da **arquitetura** — e é o que distingue o plano de um agente do de um serviço convencional.

Parte 5

Objetivos, sinais e cortes

SLIs e SLOs do agente

Família	Indicadores	Mede
operacional	latência por execução, término normal, custo por execução	*como* respondeu
de qualidade	proporção de execuções corretas, por avaliação amostrada	*se* respondeu certo

- O SLO de qualidade só é conferível com o **juiz validado** na aula de *evals*.
- O **ganho prometido na Parte 1** é um SLO declarado antes de o sistema existir; a conferência da promessa é a medida dele.
- Ganho que não se exprime como SLI mensurável é **promessa que não se pode conferir**.

Sinais, cortes e o carimbo composto

Forma da pergunta	Sinal
quanto, com que frequência	métrica — painel e alerta
o que aconteceu nesta execução	*trace* — diagnóstico
com que conteúdo	*log* ou evento
estava certo	avaliação

- **Cortes** derivam do que varia: ferramenta, agente, fatia da suíte, versão.
- **Versão = carimbo composto**: código, prompt, modelo e índice mudam independentemente. Só a versão do código não separa a mudança do *registry*, a troca do modelo pelo provedor nem a reindexação.
- Atributo de corte em métrica tem **domínio fechado**: o término é enumeração; o identificador da despesa vai no *span*.

A matriz de rastreabilidade

Uma linha por pergunta: **sinal** que a responde, **atributos** de que depende e **consumidor** — painel, alerta ou consulta. Lida nas duas direções:

- **pergunta sem sinal** — lacuna de instrumentação;
- **sinal sem pergunta** — custo sem uso, e sai.

Pergunta	Sinal	Atributos	Consumidor
quantas análises saem sem veredito?	contagem de `invoke_agent`	`agente.veredito`, versão	alerta
qual integração está falhando?	`execute_tool` com `ERROR`	`gen_ai.tool.name`	painel
alguma análise usou dado de outra?	argumentos de `execute_tool`	despesa pedida e consultada	consulta sob demanda

Parte 6

Amostragem, qualidade, ação e o que fica fora

Amostragem e qualidade

Amostragem. O desfecho de uma execução só se conhece **no fim**: guardar **todo** término anormal, erro e avaliação reprovada, e uma fração do resto — amostragem na cauda, ou decisão da aplicação ao fim da execução.

$$V = N \cdot \bar{s} \cdot \bar{b}$$

execuções por dia × *spans* por execução × bytes por *span*, contra o limite do plano. Retenção entre o mínimo da governança e o máximo da lei.

Rótulos de qualidade.

- **explícitos** — o juiz sobre uma amostra: custo por execução avaliada;
- **implícitos** — rejeição ou edição na confirmação humana, reformulação, contestação: gratuitos, mas só existem onde alguém viu.

Toda falha encontrada volta ao *golden dataset*, com a origem declarada.

Ação e exclusão

Todo alerta declara destinatário e procedimento. Alerta sem procedimento produz notificação sem ação e ensina a ignorar os legítimos.

Limiar derivado do SLO: alerta-se quando o orçamento de erro é consumido depressa demais para durar a janela — a **taxa de consumo** (*burn rate*) (BEYER et al., 2018).

O painel diagnostica, descendo pelos cortes; o alerta avisa.

O que fica fora. Prompt e resposta carregam dado pessoal e ficam **fora dos *spans*** por padrão. Quando o juiz precisa do conteúdo, ele vai a um armazenamento próprio, ligado pelo `trace_id`, com acesso e retenção declarados.

Parte 7

O plano do auditor, e outra arquitetura

O auditor: da análise às métricas

Origem no plano	Sinal resultante
SLO: p95 da execução ≤ 30 s (28 dias)	latência p50/p95
SLO: custo médio ≤ R$ 0,05 por análise	custo por execução
modo: orçamento esgotado	**distribuição dos motivos de término**
modo: regulamento indisponível	**taxa de erro por ferramenta**
modo: resposta sem veredito · SLO: ≥ 95% com término normal e veredito	`agente.veredito`
modo: conclusão errada · SLO: acerto ≥ 90%	acerto amostrado

As quatro métricas da nota sobre observabilidade aparecem como **derivação**. O veredito, que não está entre elas, entra por um modo de falha — e é por isso que o `agente.py` o grava.

Outra arquitetura, outra lista — e o plano como hipótese

Assistente de atendimento com RAG e aprovação humana, pelo mesmo método:

Modo de falha próprio	Sinal
recuperação vazia	proporção de consultas com zero trechos
trecho de versão antiga	versão do índice no *span* de recuperação
proposta rejeitada ou editada	taxas de rejeição e edição pelo atendente — rótulo implícito em **todas** as execuções
aprovação demorada	tempo de espera, **fora** da latência do agente

O plano é hipótese até a falha ser provocada e o sinal registrá-la. E envelhece com a arquitetura: um segundo agente cria fronteiras e modos de falha novos, e o corte por agente deixa de ser trivial.